

回溯法

从本讲起开始介绍算法设计策略，将主要介绍回溯、分治、动态规划、贪心、NP 问题、近似算法、局部搜索、随机算法、并行算法以及外部排序，其中从 NP 问题到随机算法这四讲将是这门课最具理论色彩的部分，其中有无数闪烁着人类智慧光芒的伟大结论，也有留待未来的天才解决的迷人猜想。当然这也意味着这几讲的内容颇具挑战性，因为事实上在具有强大的理论计算机底蕴的学校，这些内容每一章都可以用整整一门课来讲解，上千页的教材都无法完全展现其魅力，而我们只能在有限的课时中领略一些思想。

在接下来对算法设计的讨论中，有两个要求将是讨论的核心：

1. 有效性 (effectiveness)：能保证算法的结果是正确的，或者之后近似算法保证结果在一定近似比内，或者随机算法保证一定概率内是合理的；
2. 有效率 (efficiency)：算法的运行速度应当是快速的。

在之后所有的算法中都会思考这两个问题。实际上，在学习数据结构的时候也需要考虑操作保持数据结构的结构和序性质，以及希望操作是比较快速的，因此这些目标都是有共通性的。

本讲从思想最简单、看起来不那么应该出现在高级算法设计课程中的回溯法开始。接下来围绕具体算法说明这些思想，并给出理解过程所需的伪代码与证明；理论章节中也会补充更多背景，供希望深入理解这些有趣但具有挑战性的专题的同学阅读。

6.1 基本思想

有些问题要求我们给出符合条件的解，而所有的可能性是有限的，这时可以通过所谓的暴力搜索 (exhaustive search)，或称蛮力法 (Brute Force) 来对所有的可能性逐一检查得到正确的解。这样的方法显然可以保证算法有效性，因为所有可能都被检查了，然而在很多问题中，所有可能性的个数（搜索空间）可能是相当巨大的，这样的算法显然不是很有效率。

然而在很多情况下，我们可以很轻松地通过一些判断将很大一部分可能的结果排除在正确答案之外，这样的排除过程我们称为“剪枝” (pruning)，至于为什么叫这个名字，是因为经常会将搜索正确答案的过程用一棵树表示，我们在八皇后问题中就能看清这一点。

6.2 八皇后问题

在 8×8 棋盘上放八个皇后，要求任意两个皇后不在同一行、同一列或同一条对角线上。把第 i 行皇后所在的列记为 x_i ，一个摆法就表示成 $(x_1, \dots, x_8)$ 。这样表示时，每一行都恰好放了一个皇后，因此不会出现同行的冲突。如果暂时不考虑其他限制，每个 x_i 都有 8 种选择，一共有 8^8 种摆法。再考

考虑不能同列的要求： $x_1, \dots, x_8$ 必须互不相同，也就是把 $1, \dots, 8$ 排成一个排列，所以只剩下 $8!$ 种可能。最后还要检查对角线：第 i 行和第 j 行的两个皇后在同一条对角线上，当且仅当 $|x_i - x_j| = |i - j|$ ，因为此时它们相隔的行数与列数相等。

我们可以用一棵树表示逐行放置皇后的过程。根结点表示空棋盘；从根出发选择一条边，就确定了第一行皇后的位置，走到下一层再确定第二行的位置，以此类推。因此，深度为 r 的结点表示前 r 行皇后已经放好，从根走到深度为 8 的叶子，就得到一种完整摆法。假如前两个皇后已经在同一条对角线上，继续放置其他皇后也无法消除这个冲突，所以这个结点下面的所有摆法都不用再检查，这就是剪枝。

先用规模较小的四皇后问题看看搜索怎样进行。图 6.1 画出了 $4! = 24$ 个排列，以及逐行选择各列的过程。假设第一行的皇后放在第 1 列，第二行就有第 2、3、4 列三种选择：放在第 2 列会立即产生对角线冲突；放在第 3 列后，第三行无论选剩下的第 2 列还是第 4 列，也都会产生冲突；放在第 4 列后，第三行只能选第 2 列，但第四行剩下的第 3 列又与第三行冲突。因此，第一行选第 1 列时没有解。接下来撤回这个选择，把第一行的皇后改放到第 2 列，继续搜索就能找到 $(2, 4, 1, 3)$ 。同样地，第一行选第 3 列时能找到另一个解 $(3, 1, 4, 2)$ ，两个解的棋盘摆法见图 6.2。

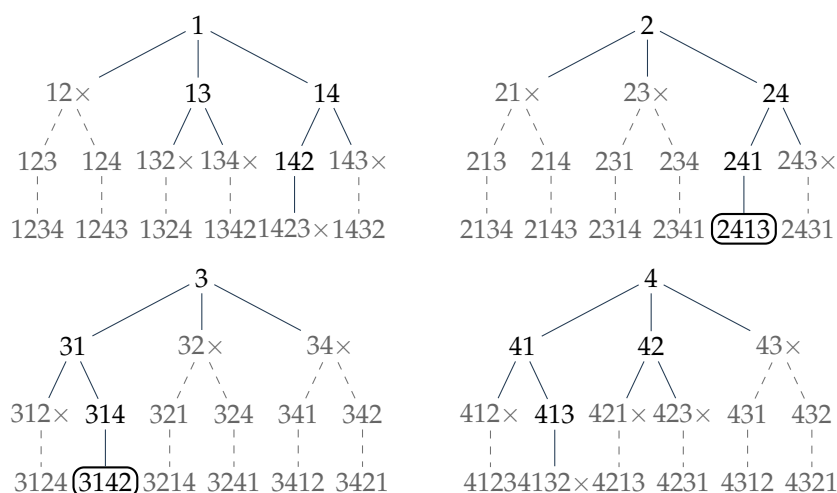


图 6.1 四皇后的排列树，叉号标出首次出现对角线冲突的位置，其下虚线部分可以跳过，方框标出两个解。

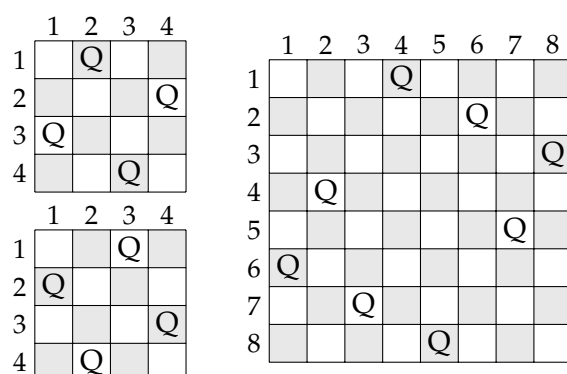


图 6.2 左列为四皇后的两个解，右列为一个八皇后解。行从上到下、列从左到右编号。

对于 n 皇后问题，可以用三个布尔数组分别记录哪些列、哪些从左上到右下的对角线、哪些从右上到左下的对角线上已经有皇后。考虑一个格子 (r, c) ，它所在的列编号为 c ，因此可以通过数组

$col[c]$ 记录并检查这一列是否已经有皇后。两条对角线的编号可以用行号 r 和列号 c 来计算。先看从左上到右下的方向, 沿这个方向走一格, 行号和列号同时增加 1, 所以差值 $r - c$ 不变。不同的这类对角线则有不同的差值, 因此可以用 $r - c$ 区分这些对角线。由于这个差值可能为负数, 我们统一加上 n , 得到范围为 $1, \dots, 2n - 1$ 的编号 $r - c + n$ 便于作为数组下标。再看从右上到左下的方向, 此时 $r + c$ 不变, 可以直接用它给这个方向的对角线编号。记两个方向分别使用数组 $diag_1$ 和 $diag_2$, 则可以通过数组 $diag_1[r - c + n]$ 和 $diag_2[r + c]$ 来记录和检查两条对角线的占用情况。

例如, 在 4×4 棋盘上, 若先把皇后放在 $(1, 3)$, 就将 $diag_1[1 - 3 + 4] = diag_1[2]$ 和 $diag_2[1 + 3] = diag_2[4]$ 置为 **true**, 其余对角线标记仍为 **false**。接下来尝试第二行: 若选 $(2, 4)$, 就会查到 $diag_1[2 - 4 + 4] = diag_1[2]$ 为 **true**, 说明它与已有皇后处于同一条从左上到右下的对角线上; 若选 $(2, 2)$, 则会查到 $diag_2[2 + 2] = diag_2[4]$ 为 **true**, 说明另一方向的对角线发生冲突。而 $(2, 1)$ 对应的 $diag_1[5]$ 和 $diag_2[3]$ 都为 **false**, 第 1 列也没有皇后, 所以可以放置。以后回溯取走某个皇后时, 再将它对应的两个对角线标记恢复为 **false**。

下面的 $Queens(r)$ 负责尝试第 r 行皇后的位置。调用它时, 前 $r - 1$ 行已经放好, 它会继续放置第 r 行及后面的皇后, 找出所有符合要求的摆法。数组 x 保存每一行皇后所在的列, 用 $x[r] = 0$ 表示第 r 行尚未放置皇后。开始搜索前, 将 $x[1..n]$ 全部置为 0, 将 $col[1..n]$ 、 $diag_1[1..2n - 1]$ 和 $diag_2[2..2n]$ 全部置为 **false**, 再调用 $Queens(1)$ 。这些数组由各层递归共用, 因此每层在尝试完一个位置后, 都要撤销自己所做的修改。

算法 6.1 枚举 n 皇后的所有解 $Queens(r)$

输入: $n \geq 1$; 当前要放置皇后的行号 r , 前 $r - 1$ 行已经合法放置

输出: 输出在此前 $r - 1$ 行的摆法下找到的所有完整解; 返回时恢复调用前的数组状态

```

1: if  $r = n + 1$  then                                     ▷ 前  $n$  行都已放好
2:   输出当前摆法  $x[1..n]$  的副本
3:   return                                               ▷ 回到上一层, 继续尝试其他位置
4: end if
5: for  $c = 1, 2, \dots, n$  do                               ▷ 依次尝试第  $r$  行的各列
6:    $a \leftarrow r - c + n$ ;  $b \leftarrow r + c$              ▷ 经过格子  $(r, c)$  的两条对角线编号
7:   if  $col[c] = \text{false}$  且  $diag_1[a] = \text{false}$  且  $diag_2[b] = \text{false}$  then
8:      $x[r] \leftarrow c$                                    ▷ 在第  $r$  行、第  $c$  列放下皇后
9:      $col[c] \leftarrow \text{true}$ ;  $diag_1[a] \leftarrow \text{true}$ ;  $diag_2[b] \leftarrow \text{true}$ 
10:     $Queens(r + 1)$                                      ▷ 保持当前皇后的位置, 继续放后面的皇后
11:     $x[r] \leftarrow 0$                                    ▷ 后续摆法已尝试完, 取走当前皇后
12:     $col[c] \leftarrow \text{false}$ ;  $diag_1[a] \leftarrow \text{false}$ ;  $diag_2[b] \leftarrow \text{false}$ 
13:   end if
14: end for

```

下面分析算法需要多少时间。先计算搜索树中可能有多少个结点, 注意其中每个结点对应一种已经放好前若干行的摆法。我们暂时不检查对角线, 只要求皇后不能同列。第一行有 n 列可选, 第二行要避开已经占用的一列, 还剩 $n - 1$ 种选择, 第三行剩 $n - 2$ 种, 以此类推。因此, 前 r 行的摆法共有 $n(n - 1) \cdots (n - r + 1) = n! / (n - r)!$ 种, 对应搜索树深度为 r 的那些结点。深度为 0 时只有表示空棋盘的根结点, 也符合这个式子。实际算法还要检查对角线, 会跳过有冲突的摆法, 所以每一

层访问的结点数只会更少。把从深度 0 到深度 n 的各层加起来，实际访问的结点总数 N 满足

$$N \leq \sum_{r=0}^n \frac{n!}{(n-r)!} = n! \sum_{j=0}^n \frac{1}{j!} < 3n!$$

其中最后一个不等式需要注意到当 $j \geq 2$ 时， $j! \geq 2^{j-1}$ 。

有了结点数，还要看处理一个结点需要多少时间。如果还没有放满 n 行，代码中的 for 循环就会依次尝试当前行的 n 列。对于每一列，计算两个对角线编号、检查三个数组位置，以及放置和撤销时修改标记，都只需常数时间。因此，不计递归调用内部的工作，处理这个结点需要 $O(n)$ 时间，递归中的工作会在对应的孩子结点处另行计入。如果已经放满 n 行，就复制并输出数组 $x[1..n]$ ，同样需要 $O(n)$ 时间。所以，访问至多 $O(n!)$ 个结点、每个结点花费 $O(n)$ 时间，包含输出在内的总时间上界就是 $O(n \cdot n!)$ 。

为了比较回溯算法中常见的几种时间界，我们需要一些基本的极限结论。设 $a > 1$ 、 $b > 0$ 都是固定常数，则有

$$\lim_{n \rightarrow \infty} \frac{n!}{n^n} = \lim_{n \rightarrow \infty} \frac{a^n}{n!} = \lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0.$$

第一个极限可以把分式写成 $\prod_{i=1}^n (i/n)$ ：前 $\lfloor n/2 \rfloor$ 个因子都不超过 $1/2$ ，其余因子不超过 1，因此整个乘积趋于 0。对于第二个极限，数列 $a^n/n!$ 的后一项与前一项之比为 $a/(n+1)$ ；当 n 足够大时，这个比值不超过 $1/2$ ，所以数列也趋于 0。第三个数列的相邻项之比为 $(1+1/n)^b/a$ ，趋于小于 1 的常数 $1/a$ ，同样可以用等比数列估计并得到极限为 0。因此，固定次数的多项式比固定底数的指数增长慢，后者又比阶乘增长慢，而阶乘比 n^n 增长慢。当然有时候可能需要更精确的阶乘估计，此时我们可以使用斯特林公式 $n! \sim \sqrt{2\pi n}(n/e)^n$ 。

6.3 收费公路问题

假设一条笔直的公路上有若干个出口，有人测出了每两个出口之间的距离，却没有记下每个距离对应哪两个出口。我们能否只根据这些距离信息还原出口的位置？这就是收费公路重建问题 (turnpike reconstruction problem)。

例如，图 6.3 中有六个出口，位置分别为 0, 3, 5, 6, 8, 10，单位为千米。每两个出口提供一个距离，所以一共有 $\binom{6}{2} = 15$ 个距离。将它们从小到大排列，就得到图中的距离集合 D 。我们的任务就是已知集合 D ，反推出口在公路上的位置。

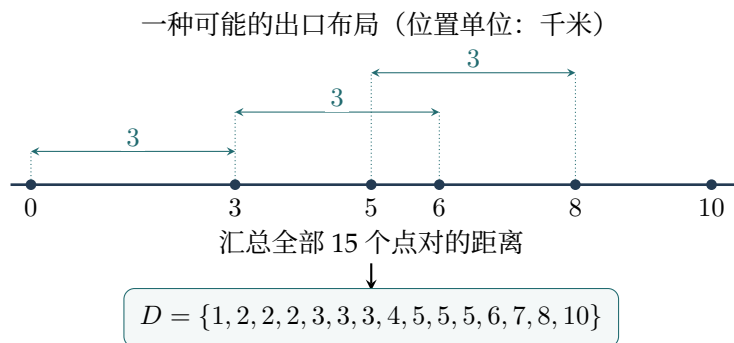


图 6.3 收费公路重建问题的示例。

这里有两点需要说明。首先，同一个距离可能出现多次，例如，0 到 3、3 到 6、5 到 8 的距离都是 3，所以集合中有三个 3，这样的集合称为多重集。其次，整体平移不会改变任何两个出口之间的

距离, 因此无法仅凭距离知道出口在公路上的绝对位置, 因此我们固定最左出口的位置为 0, 只求其余出口相对于它的位置。

一般地, 若共有 n 个不同的出口, 输入就有 $n(n-1)/2$ 个正距离。如果距离个数不符合这个形式, 或者出现非正距离, 就可以直接判定无解。对于上面的例子, 由 $n(n-1)/2 = 15$ 得到 $n = 6$ 。最大的距离一定来自最左、最右两个出口, 所以把最大距离记为 W , 就能确定两端的位置为 0 和 W 。本例中 $W = 10$, 因此我们可以直接放好两个端点 0、10, 然后从 D 中扣去一份 10。

接下来的问题是: 怎样选择下一个位置? 注意此时剩余的最大距离为 8, 事实上我们只需尝试位置 8 和 $10 - 8 = 2$: 前者到左端点 0 的距离为 8, 后者到右端点 10 的距离为 8。一般地, 若剩余最大距离为 d , 两个候选位置就是 d 和 $W - d$ 。为什么最大剩余距离一定能对应到一个端点? 先从图中的布局理解。对于尚未确定的点 8, 它到左侧任意一点的距离都不超过它到 0 的距离, 到右侧任意一点的距离都不超过它到 10 的距离, 因此至少有一个端点是离它最远的点。在搜索的中间过程, 已经放好的点之间的距离都已扣除, 所以剩余距离分为两种: 未放点到已放点的距离, 以及两个未放点之间的距离。只要当前已放点的位置没有选错, 就可以对其余点继续使用上面的观察: 每个未放点到其他点的距离, 都不会超过它到某个端点的距离, 而它到两端的距离也都还在集合中。因此, **最大的剩余距离, 一定也出现在某个未放点与一个端点之间**。这个距离若是到 0 的距离, 未放点的位置就是 d ; 若是到 W 的距离, 位置就是 $W - d$ 。把这两个位置都尝试一遍, 就至少能选到一个正确的位置, 继续寻找其余点。

一个自然的问题是, 如何判断候选位置能不能放? 自然地, 新点到每个已放点的距离, 都必须在剩余集合中找到, 而且出现次数要足够。仍看本例, 尝试在 8 处放点时, 它到 0、10 的距离分别为 8、2, 剩余集合中都有, 所以可以先放下点 8, 并扣去这两个距离。此时 $X = \{0, 8, 10\}$, $D = \{1, 2, 2, 3, 3, 3, 4, 5, 5, 5, 6, 7\}$, 其中 X 表示已经放好的点集。通过这一步检查, 只能说明目前没有发现矛盾, 接下来仍需继续寻找其他点。一般地, 尝试位置 y 时, 计算它到所有已放点的距离 $\Delta(y, X) = \{|y - x| : x \in X\}$, 并按多重集统计出现次数。如果每种距离在 D 中都有足够的份数, 就从 D 扣去这些距离, 把 y 加入 X , 再递归寻找下一个点。如果后续无法放完所有点, 就撤销这次选择: 移去 y , 把刚才扣去的距离原样加回, 然后尝试另一个候选位置。

下面接着从 $X = \{0, 8, 10\}$ 往下搜索:

1. 先尝试点 7, 剩余最大距离为 7, 所以候选位置为 7 和 3。先放点 7, 它到 0, 8, 10 的距离分别为 7, 1, 3, 都能从 D 中扣去, 扣去后剩下 $D = \{2, 2, 3, 3, 4, 5, 5, 5, 6\}$ 。此时还缺两个点, 最大剩余距离为 6, 所以下一步只能尝试位置 6 或 4。放点 6 需要它到点 7 的距离 1, 但清单中已经没有 1; 放点 4 时, 它到 0 和 8 的距离都是 4, 需要两份 4, 清单中却只有一份。两个候选都失败, 说明先放点 7 这条路走不通。
2. 撤销点 7, 改试点 3。从 X 移去 7, 并把距离 7, 1, 3 加回 D , 就恢复到放点 7 之前的状态。现在尝试另一个候选位置 3, 它到 0, 8, 10 的距离为 3, 5, 7, 扣去后得到 $X = \{0, 3, 8, 10\}$, $D = \{1, 2, 2, 3, 3, 4, 5, 5, 6\}$ 。
3. 继续放点 6, 最大剩余距离为 6, 候选位置为 6 和 4。先试 6, 它到 0, 3, 8, 10 的距离分别为 6, 3, 2, 4, 均可扣去, 得到 $X = \{0, 3, 6, 8, 10\}$, $D = \{1, 2, 3, 5, 5\}$ 。
4. 最后放点 5, 最大剩余距离为 5, 两个候选位置 5 和 $10 - 5$ 重合, 只需尝试一次。点 5 到 0, 3, 6, 8, 10 的距离分别为 5, 2, 1, 3, 5, 恰好用完所有剩余距离。六个点全部放好, 得到解 $\{0, 3, 5, 6, 8, 10\}$, 与开头图中的布局一致。

下面给出上述思路对应的伪代码。我们按从左到右的顺序, 用数组 $X[1..n]$ 保存各个点的位置。调用递归过程前, 先令 $X[1] = 0, X[n] = W$, 从 D 中扣去一份 W , 然后调用 $\text{Reconstruct}(X, D, n, 2, n-1)$ 。若输入为空, 则只有一个点, 直接取 $X[1] = 0$ 即可。

算法 6.2 收费公路重建 $\text{Reconstruct}(X, D, n, \text{left}, \text{right})$

输入: 已确定的两段坐标 $X[1..\text{left} - 1]$ 、 $X[\text{right} + 1..n]$, 以及剩余距离 D

输出: 有解时返回 true, 并将解保存在 X 中; 无解时返回 false, 恢复 D

```

1:  $Found \leftarrow \text{false}$ 
2: if  $D$  为空 then
3:   return true
4: end if
5:  $D_{\max} \leftarrow \text{Find\_Max}(D)$ 
6:  $OK \leftarrow \text{Check}(D_{\max})$  ▷ 检查位置是否合法、所需距离的份数是否足够
7: if  $OK$  then
8:    $X[\text{right}] \leftarrow D_{\max}$ 
9:   for  $i = 1, \dots, \text{left} - 1$  do  $\text{Delete}(|X[\text{right}] - X[i]|, D)$ 
10:  for  $i = \text{right} + 1, \dots, n$  do  $\text{Delete}(|X[\text{right}] - X[i]|, D)$ 
11:    $Found \leftarrow \text{Reconstruct}(X, D, n, \text{left}, \text{right} - 1)$ 
12:   if  $\neg Found$  then
13:     for  $i = 1, \dots, \text{left} - 1$  do  $\text{Insert}(|X[\text{right}] - X[i]|, D)$ 
14:     for  $i = \text{right} + 1, \dots, n$  do  $\text{Insert}(|X[\text{right}] - X[i]|, D)$ 
15:   end if
16: end if
17: if  $\neg Found$  then
18:    $OK \leftarrow \text{Check}(X[n] - D_{\max})$ 
19:   if  $OK$  then
20:      $X[\text{left}] \leftarrow X[n] - D_{\max}$ 
21:     for  $i = 1, \dots, \text{left} - 1$  do  $\text{Delete}(|X[\text{left}] - X[i]|, D)$ 
22:     for  $i = \text{right} + 1, \dots, n$  do  $\text{Delete}(|X[\text{left}] - X[i]|, D)$ 
23:      $Found \leftarrow \text{Reconstruct}(X, D, n, \text{left} + 1, \text{right})$ 
24:     if  $\neg Found$  then
25:       for  $i = 1, \dots, \text{left} - 1$  do  $\text{Insert}(|X[\text{left}] - X[i]|, D)$ 
26:       for  $i = \text{right} + 1, \dots, n$  do  $\text{Insert}(|X[\text{left}] - X[i]|, D)$ 
27:     end if
28:   end if
29: end if
30: return  $Found$ 

```

时间开销取决于尝试了多少种放法, 以及每次尝试要处理多少个距离。若用平衡搜索树保存每种距离及其剩余份数, 不同距离至多有 $O(n^2)$ 种, 查找、插入、删除一份距离和取得最大距离都可以在 $O(\log n)$ 时间内完成。一次尝试至多涉及 $n - 1$ 个距离, 因此检查、扣除和失败后的恢复共需

$O(n \log n)$ 时间。

由于两端点固定后还需放置 $n - 2$ 个点，所以搜索树深度至多为 $n - 2$ ，并且每次调用至多尝试两个位置，因此搜索树的结点数至多为 $1 + 2 + \dots + 2^{n-2} = O(2^n)$ 。将每个结点的处理时间相加，再计入建立距离表的 $O(n^2 \log n)$ 时间，得到总时间上界 $O(n^2 \log n + n \log n 2^n)$ 。实际搜索中，Check 会排除一部分候选位置，但上述最坏情况分析仍然是指数级的。

6.4 回溯的一般算法与搜索顺序

我们可以从皇后问题和收费公路问题中归纳出回溯的一般写法。设一个解由 n 次选择 $x_1, \dots, x_n$ 组成， S_i 是第 i 次选择的候选集合， R 表示题目要求满足的条件。调用 $\text{Backtracking}(i)$ 时，前 $i - 1$ 次选择已经确定，接下来逐一尝试 S_i 中的候选值。若 Check 发现当前选择违反了条件，就跳过它；否则先执行这个选择，再递归确定第 $i + 1$ 次选择。递归失败时撤销本次选择，继续尝试下一个候选值；递归成功时则保留答案，直接逐层返回。

算法 6.3 寻找一解的回溯框架 $\text{Backtracking}(i)$

输入： 前 $i - 1$ 次选择已确定；候选集合 S_i 和约束 R 由具体问题给出

输出： 有解时返回 true 并保留答案；无解时返回 false 并撤销本次调用的修改

```

1: Found  $\leftarrow$  false
2: if  $i > n$  then
3:   return true
4: end if
5: for 依次取  $x_i \in S_i$  do
6:    $OK \leftarrow \text{Check}((x_1, \dots, x_i), R)$ 
7:   if  $OK$  then
8:     执行选择  $x_i$ , 更新相应状态
9:      $Found \leftarrow \text{Backtracking}(i + 1)$ 
10:    if  $\neg Found$  then
11:       $\text{Undo}(i)$  ▷ 恢复到只确定前  $i - 1$  次选择的状态
12:    end if
13:  end if
14:  if  $Found$  then
15:    break
16:  end if
17: end for
18: return  $Found$ 

```

上面的算法每次确定一个变量 x_i 的值，再递归确定下一个变量 x_{i+1} 。如果这些变量可以按任意顺序确定，那么先选哪一个，会不会影响搜索的工作量？例如，要枚举三个变量的取值组合，其中 $a \in \{1, 2\}$ 、 $b \in \{1, 2, 3\}$ 、 $c \in \{1, 2, 3, 4\}$ 。既可以先选 a ，再选 b ，最后选 c ，也可以反过来先选 c 。图 6.4 画出了这两种顺序的完整搜索树（暂时不做剪枝），从根走到一个叶子，便确定了一组 (a, b, c) 。图中蓝色路径都得到 $(1, 2, 3)$ ，只是三个值被选中的先后不同。

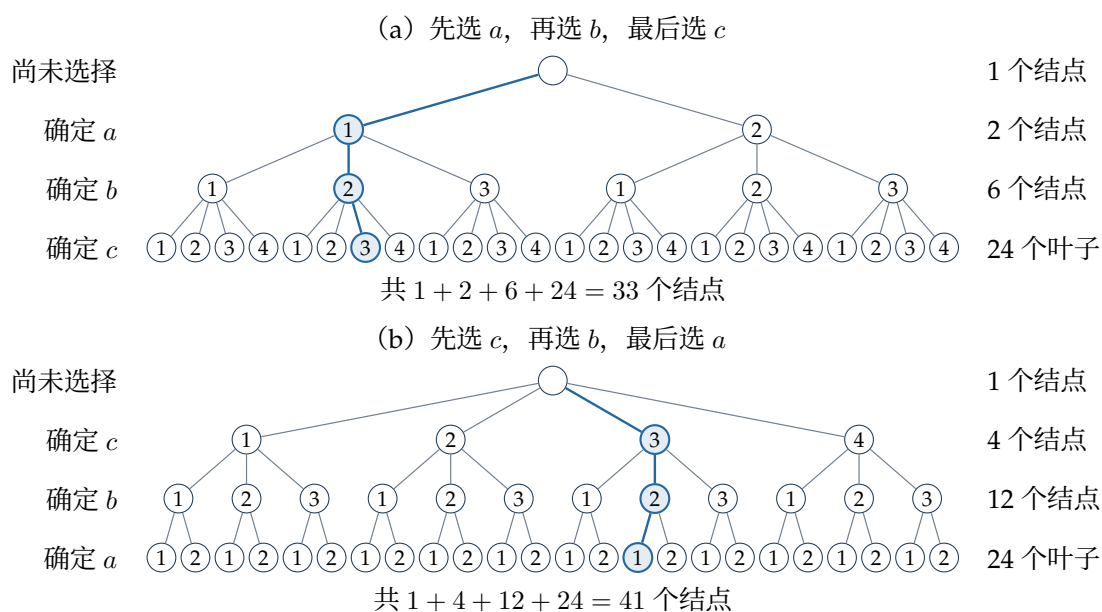


图 6.4 同样的 24 组取值, 不同的选择顺序, 非根结点中的数字表示本步选定的值。

不难理解, 两种顺序枚举的完整组合完全相同 (24 个), 但中间需要处理的结点数不同: 上图只有 $2 + 6 = 8$ 个, 下图有 $4 + 12 = 16$ 个。算上根和叶子, 两棵树分别有 33 和 41 个结点。因此不难看出, **先处理候选值较少的变量, 可以减少这类中间结点的数量**。实际回溯还会根据约束排除不合法的取值, 因此也常优先确定当前合法候选值较少的变量。例如, 一个变量已经没有任何合法取值, 就能立即判定当前分支无解。

6.5 博弈

井字棋 (tic-tac-toe) 是一个非常经典的游戏, 在 3×3 的棋盘上进行。两人轮流在空格中画圈或画叉, 先在一行、一列或一条对角线上连成三个相同符号的一方获胜。如果棋盘填满时双方都没有连成三个, 就算平局。下面让计算机画圈、人类画叉, 考虑如何编写程序, 让计算机更好地决定下一步下在哪里。

这种结果由多个参与者的选择共同决定的问题属于博弈。在井字棋中, 参与者就是计算机和人类。一方的策略规定了每一种轮到自己行动的局面下该怎样落子; 要比较策略的好坏, 还需要用数值表示对结果的偏好, 这个数值称为效用。我们把计算机获胜、平局、失败的效用分别记为 $+1, 0, -1$, 人类的效用取相反数。这样, 计算机希望分数尽可能大, 人类希望分数尽可能小, 双方效用之和始终为 0, 称为零和博弈。

一种非常直接的方法是枚举后续的所有对局。假设当前轮到计算机, 就以当前局面为根, 为计算机的每一种合法落子建立一个孩子; 接下来轮到人类, 再为人类的每一种应对建立下一层孩子。如此交替展开, 直到游戏结束, 就得到一棵博弈树。每个叶子对应一种结束的局面, 可以按计算机获胜、平局或失败, 分别标上 $+1, 0, -1$ 。当然, 在枚举了这些结果之后, 需要判断当前应该选择哪一步, 这就需要引入极小极大 (minimax) 策略。

6.5.1 极小极大策略

当我们枚举了所有结果后，假设当前计算机有两种落子选择：选择第一种后，计算机有可能赢，但人类也有可能选择让计算机输的策略；选择第二种后，计算机至少能够保住平局。即使第一种选择也有获胜的可能，计算机仍应选择第二种，因为不能把希望寄托在对手犯错上。这就是极小极大（minimax）策略的出发点，即在选择策略时，计算机认为人类总是会选择最小化（min）计算机的效用的策略，而计算机则会在人类的这一选择的基础上，选择最大化（max）自己的效用的策略。

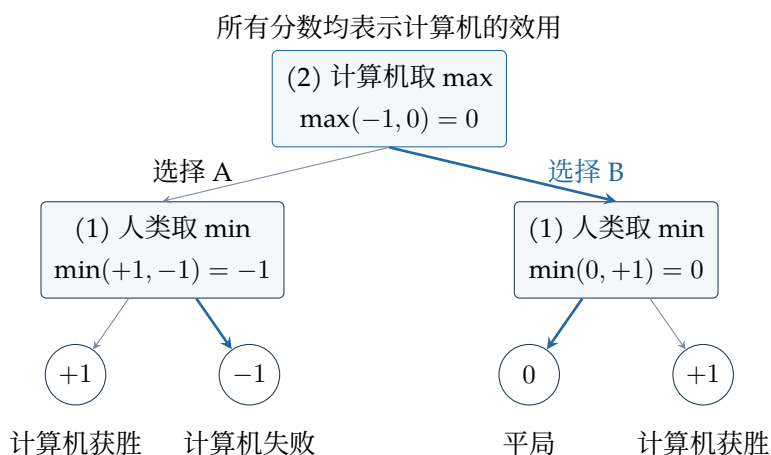


图 6.5 一棵简化的博弈树。先计算人类在每个分支中会选择的最小值，再由计算机取最大值。蓝色箭头表示相应行棋方的选择。

图 6.5 把这两种选择画成一棵简化的博弈树。若计算机选 A，人类可以在 +1 和 -1 中选择，计算机认为人类会取使计算机失败的 -1；若计算机选 B，计算机认为人类会在 0 和 +1 中取 0。因此，计算机比较的是 A 对应的 -1 和 B 对应的 0，因此应当选择 B，保证至少平局。

当然读者可能会有疑惑，为什么我们在做决策的时候需要认为对手会选择让我们最不利的决策。一方面，从直观来看，因为这是零和博弈，对手的利益与我们的利益完全相反，因此对手的最优策略就是让我们最不利的策略，此时选择 minimax 策略也能让我们至少不会“输得太惨”；另一方面，零和博弈的 minimax 策略实际上是双方的纳什均衡（Nash equilibrium），感兴趣的读者可以查阅博弈论相关资料进一步了解。

上面的算法需要一直搜索到游戏结束，才能给叶子评分。对于较复杂的游戏（如国际象棋、围棋等），后续对局太多，往往无法在合理的时间内搜索到终局。为此，我们可以只搜索到一定深度，遇到未结束的局面就用一些启发式的估值函数给它们打分。仍以井字棋为例，获胜需要占满一行、一列或一条对角线，棋盘上一共有八条这样的直线。如果某条线上已经有了对方的棋子，自己就无法再通过这条线获胜。因此，可以统计双方各自还剩多少条没有被对方占用的直线，用它们的数量差来衡量当前局面。记局面 P 中不含人类棋子的直线数为 $W_C(P)$ ，不含计算机棋子的直线数为 $W_H(P)$ ，取

$$f(P) = W_C(P) - W_H(P).$$

按照这个估值，计算机可用的直线比人类多，分数就为正；比人类少，分数就为负。图 6.6 中，人类的叉位于上边中间，经过这个格子的第一行和第二列都不能再由计算机占满，所以 $W_C(P) = 8 - 2 = 6$ ；计算机的圈位于中心，经过中心的一行、一列和两条对角线都不能再由人类占满，所以 $W_H(P) = 8 - 4 = 4$ ，于是 $f(P) = 6 - 4 = 2$ 。直观而言，这表明计算机在这个局面下比人类有更多的获胜机会。总而言

之，我们可以基于这一思想展开部分博弈树，使用启发式函数对叶子结点进行评分，然后再使用极小极大策略选择下一步。

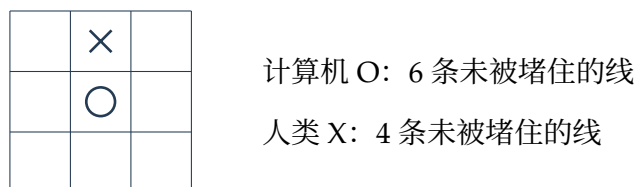


图 6.6 估值函数计算例子。

6.5.2 $\alpha - \beta$ 剪枝

在遍历博弈树计算 minimax 策略的过程中，我们不一定需要遍历所有的分支。我们把轮到计算机的结点记为 MAX，轮到人类的结点记为 MIN。图 6.7 左边的根是 MAX，左孩子的值已经算出为 44。右孩子是 MIN，刚算出它的一个孩子值为 40，就能知道它最终的值至多为 40：后面即使出现更大的值，MIN 仍会选择 40；如果出现更小的值，它只会更低。因此，根一定可以选择左边的 44，无需继续搜索右侧 MIN 的其他孩子。这称为 α 剪枝。

图的右边则相反。根是 MIN，左孩子的值已经算出为 44。右孩子是 MAX，只要找到一个值为 68 的孩子，就能保证它最终的值至少为 68。根要取较小值，已经有了 44，就不必再算右侧 MAX 的其他孩子。这称为 β 剪枝。

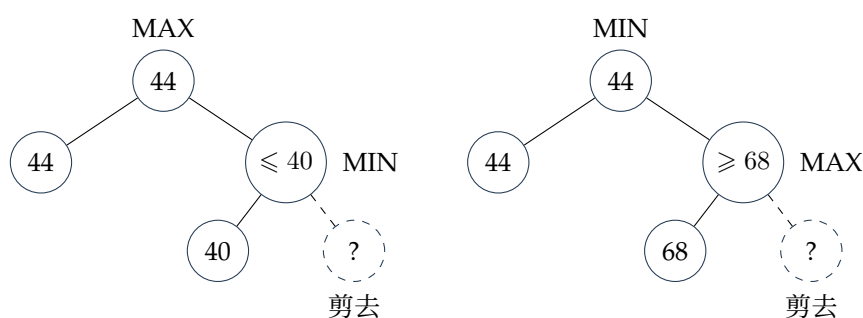
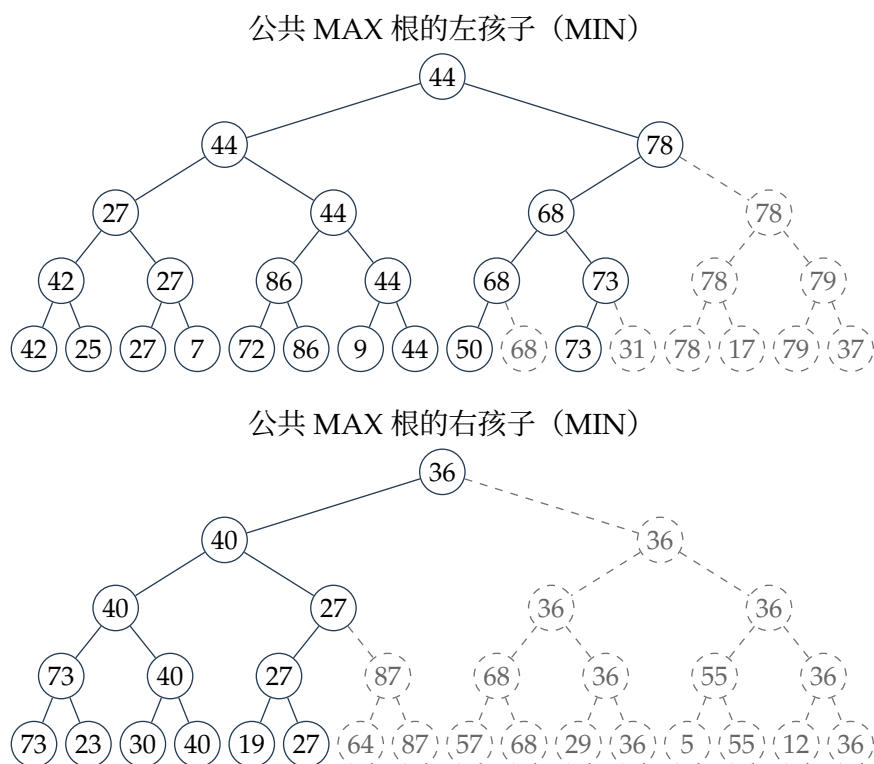


图 6.7 左图在 MIN 结点进行 α 剪枝，右图在 MAX 结点进行 β 剪枝。下方数字表示已经求得的分支值，问号表示尚未访问的分支。

图 6.8 给出了一棵较完整的博弈树。为便于排版，公共 MAX 根没有画出，上、下两部分分别是它的左、右子树。图中每个数字都是完整 minimax 搜索算出的值，灰色虚线标出 $\alpha - \beta$ 剪枝会跳过的部分，读者可以根据 α 和 β 剪枝的定义，验证图上剪枝的合理性。例如上图中的叶子 68 被剪枝的原因在于，在搜索完 50 这个叶子后，其父结点是 MAX 结点，因此它的值至少为 50。注意 50 已经大于 44，因此无论 68 这个叶子结点的取值如何，都不会影响根结点 44 的取值。

值得一提的是， $\alpha - \beta$ 剪枝中利用上下界排除分支的思想，也是整数规划的分支定界（branch-and-bound）法的基本思想，感兴趣的读者可以查阅相关资料进一步了解。此外，除了上述剪枝方法外，另一种应对巨大搜索空间的办法是随机模拟。对每个候选着法，按照事先选定的策略模拟多局后续对局，计算平均收益或获胜比例，再据此比较着法。这是蒙特卡罗方法在博弈中的一种应用。

图 6.8 $\alpha - \beta$ 剪枝示例。

6.6 习题

习题 6.1 (五皇后的角点). 在五皇后的所有合法摆法中, 至少有一个皇后位于棋盘角点的摆法有几种?

习题 6.2 (闭合马巡游). 在 $B \times B$ 的国际象棋棋盘上, 马从 (R, C) 可以跳到棋盘内的 (R', C') , 当且仅当 $(|R - R'|, |C - C'|) \in \{(2, 1), (1, 2)\}$. 一次闭合马巡游要让马恰好访问每个方格一次, 并在访问最后一个方格后再跳回起点。请回答:

1. 为什么 B 为奇数时不存在闭合马巡游?
2. 怎样用回溯法寻找闭合马巡游? 除了检查方格是否访问过, 还可以用哪些必要条件剪枝?

习题 6.3 (重建与唯一性). 考虑距离多重集 $D = \{1, 2, 2, 3, 3, 4, 5, 6, 6, 8\}$, 固定最左点为 0, 能否找到包含点 3 的解? 能否找到包含点 5 的解? 据此说明: 固定左端点后, 解是否一定唯一? 另外, 如果搜索过程中没有因为后续失败而撤销已经放好的点, 使用 AVL 树保存距离时, 为什么可以在 $O(n^2 \log n)$ 时间内完成重建?

习题 6.4 (剪枝条件). 对图 6.9 中的两棵博弈树, 均按从左到右的顺序搜索, 图中的 b 为实数, d 是右侧尚未访问的内部结点。求使以 d 为根的整棵子树被剪去的 b 的取值范围。

习题 6.5 (策梅洛定理). **策梅洛定理 (Zermelo's Theorem)** 指出, 在任何博弈树有限、具有完全信息, 结果为胜、负或平局的确定性双人零和博弈中, 以下三种情况恰好有一种成立: (1) 第一个玩家 (通常是先手) 能够保证获胜; (2) 第二个玩家 (通常是后手) 能够保证获胜; (3) 双方各自都有能够保证不输的策略。请你证明这一定理在井字棋游戏的背景下成立 (事实上证明思想可以直接推广到一般的确定性双人零和博弈)。

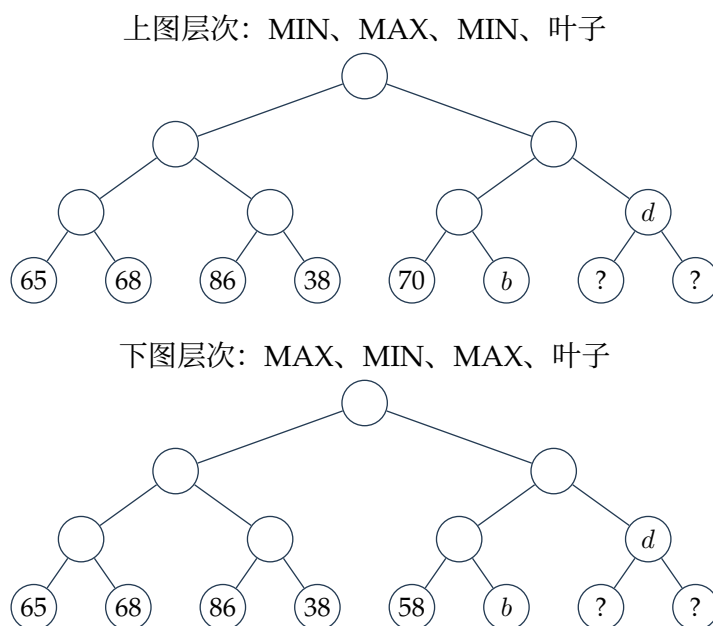
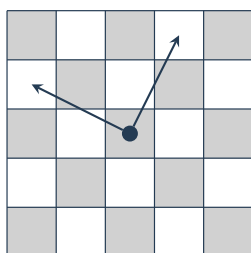


图 6.9 两棵从左到右搜索的博弈树。请判断何时可以在访问 d 之前剪去以它为根的整棵子树。

6.7 参考答案

习题 6.1. 8 种。先数左上角 $(1, 1)$ 放有皇后的摆法, 得到 $(1, 3, 5, 2, 4)$ 和 $(1, 4, 2, 5, 3)$ 。将棋盘旋转, 可以把左上角移到其他任意一个角点, 所以每个角点对应的解数都为 2, 并且不会重复。因此总数为 $4 \times 2 = 8$ 。□

习题 6.2. 先把棋盘按 $R + C$ 的奇偶性涂成黑白两色。马每跳一步, 一个坐标改变 1, 另一个改变 2, 因此 $R + C$ 的奇偶性改变, 落点的颜色也随之改变。闭合马巡游沿着黑、白交替的方格回到起点, 所以经过的黑格与白格必须一样多。若 B 为奇数, 两种颜色的方格数相差 1, 就无法在各访问一次后回到起点。因此, 奇数边长的棋盘不存在闭合马巡游。



每次马步改变方格颜色

5×5 棋盘: 两色分别 13 格、12 格

图 6.10 棋盘染色示意图。

设计回溯算法时, 可以把棋盘变成一张无向图: 每个方格对应一个顶点, 一步马跳可以到达的两个方格之间连边。我们要找的是经过全部顶点、每个顶点恰好一次的环。起点可以任取为 s , 因为这样的环一定也经过 s 。有两个剪枝的原则: (1) 如果未访问顶点加上起点构成的图不连通, 就无法形成环; (2) 如果某个未访问顶点在剩余图中不足两个邻点, 就无法进入后再离开它。□

习题 6.3. 不难得到一个解是 $\{0, 2, 3, 6, 8\}$, 其中包含 3, 然后将这个点集左右翻转, 也就是把每个坐标 x 换成 $8 - x$, 得到 $\{0, 2, 5, 6, 8\}$, 距离不变, 所以也有包含点 5 的解, 这也说明了固定左端点后, 解不一定唯一。

最后分析不需要撤销已放点的情况。已经放好 k 个点时，一个候选位置需要检查到这 k 个点的距离；至多检查两个候选位置，再扣除选中位置所需的距离，共有 $O(k)$ 次距离操作。把各层的工作相加，得到 $O(1 + 2 + \cdots + (n - 1)) = O(n^2)$ 次操作。用 AVL 树保存距离，每次操作需要 $O(\log n)$ 时间，加上建立距离表的时间，总计 $O(n^2 \log n)$ 。□

习题 6.4. 上图 $b \geq 65$ ，下图 $b \leq 68$ 。□

习题 6.5. 从任意合法局面出发，井字棋都至多再下九步就会结束，所以可以用 minimax 从终局逐层向上计算。仍把计算机获胜、人类获胜、平局分别记为 $+1, -1, 0$ 。若值为 $+1$ ，计算机可以保证获胜：在 MAX 结点，最大值为 $+1$ ，至少有一个孩子也是 $+1$ ，计算机就选择它；在 MIN 结点，最小值为 $+1$ ，所有孩子都只能是 $+1$ ，所以人类无论怎么选，都无法改变这个值。这样一直走到终局，分数仍为 $+1$ ，即计算机获胜。值为 -1 时同理：人类在 MIN 结点选择值为 -1 的孩子，而值为 -1 的 MAX 结点的所有孩子也都为 -1 ，所以人类能够保证获胜。若值为 0 ，也可以同理构造出双方都能保证不输的策略。

还需说明三种情况互不重叠，这是很显然的：一方有必胜策略时，另一方不可能有保证不输的策略，反之亦然，因此三种情况恰好有一种成立。□

6.8 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第六讲。
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*, 第二版, 第 10.5 节。